TESTING DE SOFTWARE

Challenge Técnico SDET

API de Petstore y UI de Buggy Cars, automatizadas con Cypress y ejecutadas en CI

Virginia Hanun Noviembre 2026

github.com/VirHanun5/testing-challenge-VIR

Qué se hizo

Las 3 partes del challenge y los 4 bonus, en un solo repositorio.

6

tests de API estables

Ciclo de vida, negativos, schema y limpieza. Más 7 hallazgos documentados.

8

tests de UI

Los 5 flujos pedidos, con Page Object Model.

3

navegadores en el CI

Chrome, Firefox y Edge en cada push.

8

bugs reportados

Con pasos, severidad, prioridad y capturas marcadas.

Bonus: ✓ Colección Postman con Newman ✓ BUGS.md ✓ ESTRATEGIA.md ✓ Varios navegadores

Cómo está armado

Herramientas

- Cypress para API (cy.request) y UI
- Ajv + JSON Schema para el contrato
- Faker para datos únicos por ejecución
- Newman para la colección de Postman

Organización

- Page Object Model para la UI
- Custom commands: cy.createPet, cy.loginByApi
- Suites separadas: api, api-hallazgos, ui
- URLs fuera de los tests (cypress.config.js)

Ejecución en CI

- GitHub Actions en cada push
- Lint, API, Newman y UI en 3 navegadores
- Hallazgos en un job aparte
- Reporte Mochawesome como artifact, con capturas de los fallos

Decisiones clave

Decisión	Por qué
Hallazgos sin forzar, en un job aparte	Si la API no cumple su documentación, el test falla y se documenta; las fallas conocidas no tapan una nueva
Esperar requests, nunca tiempos fijos	El test sigue apenas llega la respuesta; el linter bloquea cy.wait(5000)
Cada test crea y limpia sus datos	Tests independientes; la limpieza corre aunque el test falle
Login por API + cy.session donde no se prueba el login	Más rápido y sin depender de otra pantalla, que ya tiene su propio test
Votos “mayor que antes”, no “+1”	El sitio es público: otra persona puede votar al mismo tiempo

Qué se encontró

API PETSTORE

7 hallazgos

La API no cumple su propia documentación.

Ejemplo (H5): modificar una mascota que no existe la crea y responde 200, en lugar de devolver 404.

Los tests esperan lo documentado y fallan: no se fuerzan.

BUGGY CARS · 8 BUGS · LOS 3 DE PRIORIDAD ALTA

BUG-08 · Privacidad

Después del Logout, el perfil sigue mostrando nombre, dirección y teléfono.

BUG-07 · Funcional

El Logout no funciona en la página del ranking.

BUG-06 · Usabilidad y seguridad

Reglas de contraseña que no se avisan y un error técnico al usuario.

Y 5 más en BUGS.md: paginador, mensajes, datos y usabilidad.

Cómo se trabajó

Técnicas ISTQB aplicadas

Técnica	Dónde
Transición de estados	Ciclo de vida de la mascota
Partición de equivalencia	Negativos de la API y login fallido
Valores límite	Paginador (BUG-01, BUG-02)
Basado en riesgo	ESTRATEGIA.md
Severidad y prioridad	BUGS.md y hallazgos

Errores propios, documentados

F-01: el test de login con campos vacíos esperaba un mensaje que la app no muestra. Se corrigió para validar el del navegador.

F-02: el test de voto esperaba 4 s y la API tardó 4,3 s. Se corrigió esperando el request, sin tiempos fijos.

Regla: nada sin evidencia. Cada bug se reprodujo antes de documentarlo.

Qué agregaría con más tiempo

→ Migrar a TypeScript (hoy el proyecto es 100 % JavaScript)

→ Pruebas de performance y de accesibilidad

→ Tests de la API de Buggy Cars (hoy page=0 da 502)

→ Reporte unificado y ejecución programada

REPOSITORIO

github.com/VirHanun5/testing-challenge-VIR

README · BUGS.md · ESTRATEGIA.md · pipeline en GitHub Actions